

Data Cleaning techniques

I. Data transformation

Data transformation involves converting data into a more suitable format for analysis. Pandas provides functions to perform operations like data type conversion, string manipulation, and date/time handling.

```
In [1]: import pandas as pd

# Sample dataset
data = {
    'Name': ['Ahmed', 'SARAH', 'Imad', 'KARIM'],
    'Age': ['25', '30', 35, '40'], # stored as strings initially
    'Join_Date': ['2021-05-21', '2020/06/15', '15-07-2019', 'April 10, 2018'] #
}
df1 = pd.DataFrame(data)
print(df1)
print("\n =====")

df1['Age'] = df1['Age'].astype('int')
df1['Name'] = df1['Name'].str.capitalize()
df1['Join_Date'] = pd.to_datetime(df1['Join_Date'], format='mixed')
print(df1)
```

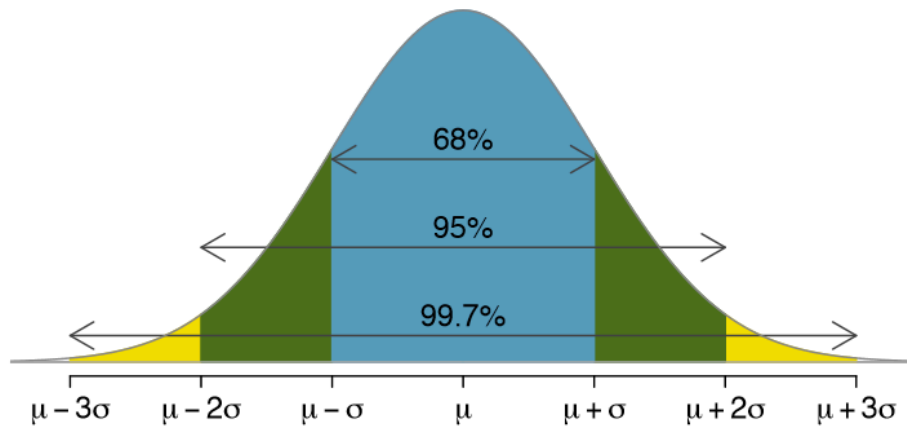
	Name	Age	Join_Date
0	Ahmed	25	2021-05-21
1	SARAH	30	2020/06/15
2	Imad	35	15-07-2019
3	KARIM	40	April 10, 2018

```
=====
      Name  Age  Join_Date
0  Ahmed   25  2021-05-21
1  Sarah   30  2020-06-15
2   Imad   35  2019-07-15
3  Karim   40  2018-04-10
```

II. handling outliers

Outliers can significantly impact the analysis results. Pandas offers various techniques to identify and handle outliers, such as using statistical methods or applying domain-specific rules.

$$Z\text{-score} = |x - \text{mean}| / \text{std.deviation}$$



```
In [3]: import pandas as pd
import numpy as np
df = pd.read_csv('../data/data1.csv')
# Identify outliers using z-scores
z_scores = np.abs(df['Duration'] - df['Duration'].mean()) / df['Duration'].std()
outliers = df[z_scores > 3]
print(df)
print(outliers)
```

	Duration	Date	Pulse	Maxpulse	Calories
0	1	'2020/12/01'	110	130	409.1
1	60	'2020/12/02'	117	145	479.0
2	60	'2020/12/03'	103	135	340.0
3	45	'2020/12/04'	109	175	282.4
4	45	'2020/12/05'	117	148	406.0
5	60	'2020/12/06'	102	127	300.0
6	60	'2020/12/07'	110	136	374.0
7	450	'2020/12/08'	104	134	253.3
8	30	'2020/12/09'	109	133	195.1
9	60	'2020/12/10'	98	124	269.0
10	60	'2020/12/11'	103	147	329.3
11	60	'2020/12/12'	100	120	250.7
12	60	'2020/12/12'	100	120	250.7
13	60	'2020/12/13'	106	128	345.3
14	60	'2020/12/14'	104	132	379.3
15	60	'2020/12/15'	98	123	275.0
16	60	'2020/12/16'	98	120	215.2
17	60	'2020/12/17'	100	120	300.0
18	45	'2020/12/18'	90	112	NaN
19	60	'2020/12/19'	103	123	323.0
20	45	'2020/12/20'	97	125	243.0
21	60	'2020/12/21'	108	131	364.2
22	45	NaN	100	119	282.0
23	60	'2020/12/23'	130	101	300.0
24	45	'2020/12/24'	105	132	246.0
25	60	'2020/12/25'	102	126	334.5
26	60	2020-12-26	100	120	250.0
27	60	'2020/12/27'	92	118	241.0
28	60	'2020/12/28'	103	132	NaN
29	60	'2020/12/29'	100	132	280.0
30	60	'2020/12/30'	102	129	380.3
31	60	'2020/12/31'	92	115	243.0
	Duration	Date	Pulse	Maxpulse	Calories
7	450	'2020/12/08'	104	134	253.3

Rfreplace outliers with a specific value

```
In [11]: # Replace outliers with a specific value
df = pd.read_csv('../data/data1.csv')
# print(df)
# Identify outliers using z-scores
z_scores = np.abs(df['Duration'] - df['Duration'].mean()) / df['Duration'].std()

df.loc[z_scores > 3, 'Duration'] = int(df['Duration'].mean())
print(df)
```

	Duration	Date	Pulse	Maxpulse	Calories
0	1	'2020/12/01'	110	130	409.1
1	60	'2020/12/02'	117	145	479.0
2	60	'2020/12/03'	103	135	340.0
3	45	'2020/12/04'	109	175	282.4
4	45	'2020/12/05'	117	148	406.0
5	60	'2020/12/06'	102	127	300.0
6	60	'2020/12/07'	110	136	374.0
7	66	'2020/12/08'	104	134	253.3
8	30	'2020/12/09'	109	133	195.1
9	60	'2020/12/10'	98	124	269.0
10	60	'2020/12/11'	103	147	329.3
11	60	'2020/12/12'	100	120	250.7
12	60	'2020/12/12'	100	120	250.7
13	60	'2020/12/13'	106	128	345.3
14	60	'2020/12/14'	104	132	379.3
15	60	'2020/12/15'	98	123	275.0
16	60	'2020/12/16'	98	120	215.2
17	60	'2020/12/17'	100	120	300.0
18	45	'2020/12/18'	90	112	NaN
19	60	'2020/12/19'	103	123	323.0
20	45	'2020/12/20'	97	125	243.0
21	60	'2020/12/21'	108	131	364.2
22	45	NaN	100	119	282.0
23	60	'2020/12/23'	130	101	300.0
24	45	'2020/12/24'	105	132	246.0
25	60	'2020/12/25'	102	126	334.5
26	60	2020-12-26	100	120	250.0
27	60	'2020/12/27'	92	118	241.0
28	60	'2020/12/28'	103	132	NaN
29	60	'2020/12/29'	100	132	280.0
30	60	'2020/12/30'	102	129	380.3
31	60	'2020/12/31'	92	115	243.0

III. Data filtering

Data filtering is the process of selecting a subset of data based on specific criteria. Pandas provides powerful filtering capabilities using boolean indexing and conditional statements.

```
In [ ]: import pandas as pd
# Random wage dataset
data = {
    'Person': ['Alex', 'Bella', 'Chris', 'Dana', 'Eli', 'Faye', 'Gabe', 'Hana',
    'Gender': ['Man', 'Woman', 'Man', 'Woman', 'Man', 'Woman', 'Man', 'Woman',
```

```

    'Age': [28, 35, 51, 31, 27, 39, 45, 33, 52, 36],
    'Job_Role': ['Data Analyst', 'Graphic Designer', 'Software Engineer', 'Data
                'Data Analyst', 'Graphic Designer', 'Data Analyst', 'Software E
                'Software Engineer', 'Data Analyst'],
    'Wage': [3200, 2900, 4500, 3100, 3700, 2800, '4100', 3300, 3900, 3000],
    'Join_Date': ['2022-01-10', '2021-12-15', '2020-06-20', '2019-11-01',
                  '2021-03-25', '2020-08-14', '2018-04-10', '2017-09-19',
                  '2022-07-30', '2019-01-05']
}

df2 = pd.DataFrame(data)
df2['Wage'] = df2['Wage'].astype(int)
df2['Age'] = df2['Age'].astype(int)
df2['Join_Date'] = pd.to_datetime(df2['Join_Date'])
print(df2)
print("\n=====")
df_sorted=df2.sort_values(by='Wage', ascending=False, na_position='first')
print(df_sorted)

print("\n=====")
# Filter rows based on a condition
filtered_df = df2[df2['Age'] > 50]

# Filter rows based on multiple conditions
filtered_df = df2[(df2['Age'] > 50) & (df2['Gender'] == 'Man')]
print(filtered_df)

print("\n=====")
df_grouped = df2.groupby('Gender')['Wage'].min()
print(df_grouped)

```

	Person	Gender	Age	Job_Role	Wage	Join_Date
0	Alex	Man	28	Data Analyst	3200	2022-01-10
1	Bella	Woman	35	Graphic Designer	2900	2021-12-15
2	Chris	Man	51	Software Engineer	4500	2020-06-20
3	Dana	Woman	31	Data Analyst	3100	2019-11-01
4	Eli	Man	27	Data Analyst	3700	2021-03-25
5	Faye	Woman	39	Graphic Designer	2800	2020-08-14
6	Gabe	Man	45	Data Analyst	4100	2018-04-10
7	Hana	Woman	33	Software Engineer	3300	2017-09-19
8	Ivan	Man	52	Software Engineer	3900	2022-07-30
9	Julia	Woman	36	Data Analyst	3000	2019-01-05

```
=====
```

	Person	Gender	Age	Job_Role	Wage	Join_Date
2	Chris	Man	51	Software Engineer	4500	2020-06-20
6	Gabe	Man	45	Data Analyst	4100	2018-04-10
8	Ivan	Man	52	Software Engineer	3900	2022-07-30
4	Eli	Man	27	Data Analyst	3700	2021-03-25
7	Hana	Woman	33	Software Engineer	3300	2017-09-19
0	Alex	Man	28	Data Analyst	3200	2022-01-10
3	Dana	Woman	31	Data Analyst	3100	2019-11-01
9	Julia	Woman	36	Data Analyst	3000	2019-01-05
1	Bella	Woman	35	Graphic Designer	2900	2021-12-15
5	Faye	Woman	39	Graphic Designer	2800	2020-08-14

```
=====
```

	Person	Gender	Age	Job_Role	Wage	Join_Date
2	Chris	Man	51	Software Engineer	4500	2020-06-20
8	Ivan	Man	52	Software Engineer	3900	2022-07-30

```
=====
```

Gender

Man	3880.0
Woman	3020.0

Name: Wage, dtype: float64

IV. Handling categorical Data

Categorical data represents distinct categories or groups. Pandas offers tools to work with categorical data, such as encoding categorical variables and performing operations like grouping and aggregation.

`dummy` : Convert categorical variable into dummy/indicator variables.

Each variable is converted in as many 0/1 variables as there are different values. Columns in the output are each named after a value; if the input is a DataFrame, the name of the original variable is prepended to the value.

```
In [1]: import pandas as pd
# Random wage dataset
data = {
    'Person': ['Alex', 'Bella', 'Chris', 'Dana', 'Eli', 'Faye', 'Gabe', 'Hana',
    'Gender': ['Man', 'Woman', 'Man', 'Woman', 'Man', 'Woman', 'Man', 'Woman',
    'Age': [28, 35, 51, 31, 27, 39, 45, 33, 52, 36],
    'Job_Role': ['Data Analyst', 'Graphic Designer', 'Software Engineer', 'Data
    'Data Analyst', 'Graphic Designer', 'Data Analyst', 'Software E
    'Software Engineer', 'Data Analyst'],
```

```

    'Wage': [3200, 2900, 4500, 3100, 3700, 2800, '4100', 3300, 3900, 3000],
    'Join_Date': ['2022-01-10', '2021-12-15', '2020-06-20', '2019-11-01',
                  '2021-03-25', '2020-08-14', '2018-04-10', '2017-09-19',
                  '2022-07-30', '2019-01-05']
}

df3 = pd.DataFrame(data)
df3['Wage'] = df3['Wage'].astype(int)
df3['Age'] = df3['Age'].astype(int)
df3['Join_Date'] = pd.to_datetime(df3['Join_Date'])

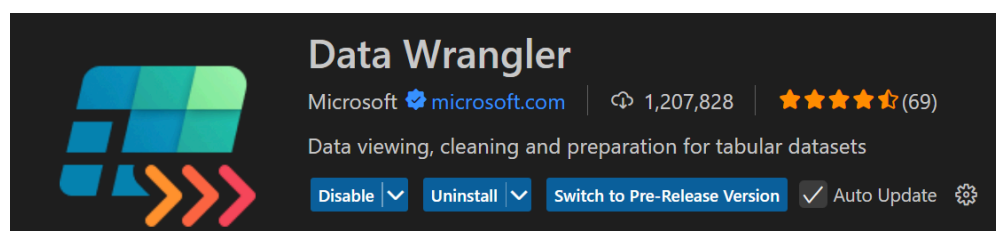
# Convert a column to categorical data type
df3['Job_Role'] = df3['Job_Role'].astype('category' )

# Encode categorical data
encoded_df = pd.get_dummies(df3, columns=['Job_Role'] )
print(encoded_df)

```

	Person	Gender	Age	Wage	Join_Date	Job_Role_Data	Analyst \
0	Alex	Man	28	3200	2022-01-10		True
1	Bella	Woman	35	2900	2021-12-15		False
2	Chris	Man	51	4500	2020-06-20		False
3	Dana	Woman	31	3100	2019-11-01		True
4	Eli	Man	27	3700	2021-03-25		True
5	Faye	Woman	39	2800	2020-08-14		False
6	Gabe	Man	45	4100	2018-04-10		True
7	Hana	Woman	33	3300	2017-09-19		False
8	Ivan	Man	52	3900	2022-07-30		False
9	Julia	Woman	36	3000	2019-01-05		True

	Job_Role_Graphic Designer	Job_Role_Software Engineer
0	False	False
1	True	False
2	False	True
3	False	False
4	False	False
5	True	False
6	False	False
7	False	True
8	False	True
9	False	False



V. Data merging and joining

Merging and joining data from multiple sources is a common task in data analysis. Pandas provides methods to combine datasets based on common columns or indexes.

```

In [ ]: import pandas as pd

# First DataFrame: Basic employee info
df1 = pd.DataFrame({

```

```

    'Employee_ID': [101, 102, 103, 104],
    'Name': ['Amin', 'Sarah', 'Karim', 'Yazid'],
    'Department': ['Sales', 'Marketing', 'HR', 'Finance']
})

# Second DataFrame: Performance review scores
df2 = pd.DataFrame({
    'Employee_ID': [101, 102, 103, 104],
    'Review_Score': [88, 92, 79, 85]
})

# Merge on the common column 'Employee_ID'
merged_df = pd.merge(df1, df2, on='Employee_ID')
print(df1)
print("\n=====")
print(df2)
print("\n=====")
print(merged_df)

# Join two DataFrames based on indexes

# First DataFrame: Employee Info
df3 = pd.DataFrame({
    'Name': ['Amin', 'Bianca', 'Carlos', 'Dina'],
    'Department': ['Sales', 'Marketing', 'HR', 'Finance']
}, index=[101, 102, 103, 104]) # Employee_ID as index

# Second DataFrame: Performance Scores
df4 = pd.DataFrame({
    'Review_Score': [88, 92, 79, 85]
}, index=[101, 102, 103, 104]) # Matching index

# Join on index
joined_df = df3.join(df4, how='inner')
# Join two DataFrames based on indexes
joined_df = df3.join(df4, how='inner' )
print(df3)
print("\n=====")
print(df4)
print("\n=====")
print(joined_df)

```

VI. Data reshaping

Data reshaping involves transforming the structure of a DataFrame, such as pivoting or unpivoting data. Pandas offers functions like melt and pivot to reshape data for better analysis.

```

In [ ]: # Unpivot (melt) data
import pandas as pd

# Wide-format data
df5 = pd.DataFrame({
    'Student_ID': ['001', '002', '003'],
    'Name': ['Ali', 'Aymen', 'Mona'],
    'Math': [85, 78, 93],

```

```

        'Science': [90, 82, 88],
        'History': [75, 85, 80]
    })

melted_df = pd.melt(
    df5,
    id_vars=['Student_ID', 'Name'], # columns to keep
    var_name='Subject',
    value_name='Score'
)
print(df5 )
print("\n=====")
print(melted_df )

# Pivot data

# Long-format data (after melt)
df6 = pd.DataFrame({
    'Student_ID': ['001', '001', '001', '002', '002', '002', '003', '003', '003'],
    'Name': ['Ali', 'Ali', 'Ali', 'Brenda', 'Brenda', 'Brenda', 'Carlos', 'Carlos', 'Carlos'],
    'Subject': ['Math', 'Science', 'History', 'Math', 'Science', 'History', 'Math', 'Science', 'History'],
    'Score': [85, 90, 75, 78, 82, 85, 93, 88, 80]
})

print("\n=====")
print(df6)
pivoted_df = df6.pivot(index='Student_ID', columns='Subject', values='Score')
print("\n=====")
print(pivoted_df)

```

You can optionally add `.reset_index()` if you want to turn `Student_ID` back into a regular column, or use `pivot_table()` to aggregate if there are duplicate entries.

```
In [ ]: pivoted_df.reset_index()
```

VII. Data imputation

Data imputation is the process of replacing missing data with substituted values. Pandas provides various imputation techniques, such as mean, median, or mode imputation, as well as more advanced methods like regression imputation.

```

In [ ]: # Mean imputation
df['column_name'] = df['column_name'].fillna(df['column_name'].mean( ) )

# Regression imputation
from sklearn.linear_model import LinearRegression
regressor = LinearRegression( )
regressor.fit(X_train, y_train)
df['column_name'] = df['column_name'].fillna(regressor.predict(X_test))

```

VIII. Data Normalization

Data normalization is a technique used to rescale data to a common range, often between 0 and 1 or -1 and 1. This can be useful for certain machine learning algorithms or when dealing with different scales of data.

```
In [ ]: import pandas as pd

# Min-max normalization
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler( )
normalized_df = pd.DataFrame(scaler.fit_transform(df), columns=df.columns)

# Standardization (z-score normalization)
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
standardized_df = pd.DataFrame(scaler.fit_transform(df), columns=df.columns)
```

IX. Data profiling

Data profiling involves summarizing and understanding the characteristics of a dataset. Pandas offers various methods to generate descriptive statistics, identify data types, and detect missing values or outliers.

```
In [46]: df7 = pd.read_csv('data/data1.csv')
# Generate descriptive statistics
print(df7.describe( ))

# Identify data types
print("\n=====")
print(df7.dtypes)

# Detect missing values
print("\n=====")
print(df7.isnull().sum( ))

# Detect duplicates
print("\n=====")
print(df7.duplicated().sum( ))
```

	Duration	Pulse	Maxpulse	Calories
count	32.000000	32.000000	32.000000	30.000000
mean	68.437500	103.500000	128.500000	304.680000
std	70.039591	7.832933	12.998759	66.003779
min	30.000000	90.000000	101.000000	195.100000
25%	60.000000	100.000000	120.000000	250.700000
50%	60.000000	102.500000	127.500000	291.200000
75%	60.000000	106.500000	132.250000	343.975000
max	450.000000	130.000000	175.000000	479.000000

```
=====
```

```
Duration      int64
Date          object
Pulse         int64
Maxpulse      int64
Calories      float64
dtype: object
```

```
=====
```

```
Duration      0
Date          1
Pulse         0
Maxpulse      0
Calories      2
dtype: int64
```

```
=====
```

```
1
```

X. Data validation

Data validation is the process of ensuring that data adheres to specific rules, constraints, or formats. Pandas provides methods to validate data and handle violations, such as raising errors or applying custom functions.

```
In [63]: data = {
    'Person': ['Alex', 'Bella', 'Chris', 'Dana', 'Eli', 'Faye', 'Gabe', 'Hana',
    'Gender': ['Man', 'Woman', 'Man', 'Woman', 'Man', 'Woman', 'Man', 'Woman'],
    'Age': [28, 35, '12', 31.6, 27, 39, 45, 33, 52, 75],
    'Job_Role': ['Data Analyst', 'Graphic Designer', 'Software Engineer', 'Data
    'Data Analyst', 'Graphic Designer', 'Data Analyst', 'Software E
    'Software Engineer', 'Data Analyst'],
    'Wage': [3200.9, 2900.8, 4500.8, 3100.4, 3700.1, 2800.5, '4100.0', 3300.2, 3
    'Join_Date': ['2022-01-10', '2021-12-15', '2020-06-20', '2019-11-01',
    '2021-03-25', '2020-08-14', '2018-04-10', '2017-09-19',
    '2022-07-30', '2019-01-05']
}

df8 = pd.DataFrame(data)

# Validate data types
df8 = df8.astype({'Age': 'int', 'Wage': 'float'})
print(df8)
print("\n =====")

# Apply custom validation function
def validate_age(age):
    if age < 18 or age > 70:
```

```

        # raise ValueError('Invalid age' )
        print(f"Invalid age found: {age}")
        return ""
    else :
        return age
df8['Age'] = df8['Age'].apply(validate_age)
print(df8)

# df8['Valid_Age'] = df8['Age'].between(18, 70)
# print(df8['Valid_Age'])

```

	Person	Gender	Age	Job_Role	Wage	Join_Date
0	Alex	Man	28	Data Analyst	3200.9	2022-01-10
1	Bella	Woman	35	Graphic Designer	2900.8	2021-12-15
2	Chris	Man	12	Software Engineer	4500.8	2020-06-20
3	Dana	Woman	31	Data Analyst	3100.4	2019-11-01
4	Eli	Man	27	Data Analyst	3700.1	2021-03-25
5	Faye	Woman	39	Graphic Designer	2800.5	2020-08-14
6	Gabe	Man	45	Data Analyst	4100.0	2018-04-10
7	Hana	Woman	33	Software Engineer	3300.2	2017-09-19
8	Ivan	Man	52	Software Engineer	3900.3	2022-07-30
9	Julia	Woman	75	Data Analyst	3000.9	2019-01-05

```

=====
Invalid age found: 12
Invalid age found: 75

```

	Person	Gender	Age	Job_Role	Wage	Join_Date
0	Alex	Man	28	Data Analyst	3200.9	2022-01-10
1	Bella	Woman	35	Graphic Designer	2900.8	2021-12-15
2	Chris	Man		Software Engineer	4500.8	2020-06-20
3	Dana	Woman	31	Data Analyst	3100.4	2019-11-01
4	Eli	Man	27	Data Analyst	3700.1	2021-03-25
5	Faye	Woman	39	Graphic Designer	2800.5	2020-08-14
6	Gabe	Man	45	Data Analyst	4100.0	2018-04-10
7	Hana	Woman	33	Software Engineer	3300.2	2017-09-19
8	Ivan	Man	52	Software Engineer	3900.3	2022-07-30
9	Julia	Woman		Data Analyst	3000.9	2019-01-05

XI. Conclusion

Data cleaning is an essential step in the data analysis process. Pandas provides a powerful and flexible toolset to handle various data cleaning tasks, from handling missing data and duplicates to data transformation, filtering, and reshaping. By mastering these techniques, you can ensure your data is accurate, consistent, and ready for meaningful analysis.